

배치 개발자 가이드

Job/Step 부터 Lease·Fencing·Checkpoint·Reconcile 까지 Batch 개발 실무 참조서

Batch 개발자·운영 개발자를 위한 실무 참조서입니다. 재시작 가능한 Batch 를 설계하고 중복·Kill·UNKNOWN 까지 안전하게 처리합니다.

- Job/Step/Policy API 를 선택합니다.
- Retry/Restart/Rerun/Reconcile 을 구분합니다.
- 다중 Worker 의 Lease/Fencing 과 Runtime 실패를 검증합니다.

목차

필요한 작업을 먼저 찾고 해당 장으로 이동합니다. 페이지 번호는 최종 PDF 기준입니다.

1 Batch 를 언제 선택할지.....	3
Public API 빠른 선택.....	3
옵션 빠른 확인.....	4
2 실행 · 중지 · 재시작 · 재처리.....	4
실행 요청 종류.....	5
Public API 빠른 선택.....	5
3 Lease · Fencing · 다중 인스턴스.....	6
동시성 보호.....	6
Public API 빠른 선택.....	6

4 Retry · Skip · Rollback 정책.....	7
옵션 빠른 확인.....	7
실패 종류별 처리.....	7
5 Reader · Writer · Processor 와 DB.....	8
설계 체크.....	8
6 Scheduler · Center-Cut · 운영 제어.....	9
운영 액션.....	9
Center-Cut 실행 전 확인.....	9
7 Batch 검증.....	10
필수 Runtime 시나리오.....	10
Batch 변경 후 검증.....	10
8 Control Plane 과 독립 Runtime.....	10
Control Plane·Scheduler·Worker.....	11
Center-Cut·Agent·Job Pack.....	11
개발용 Local Runtime 과 운영 Process.....	11
9 대량 처리·운영 인계·Source.....	11
Partition/Chunk 용량 결정.....	11
운영 인계 항목.....	11

1 Batch 를 언제 선택할지

대량·예약·재시작·분할 실행이 필요한 업무를 Batch Job/Step 구조로 설계합니다.

Public API 빠른 선택

표 1-1. Public API 빠른 선택

Public API 빠른 선택의 선택 기준과 확인 항목을 빠르게 비교합니다.

하려는 일	Public API / Annotation	핵심 옵션	언제 선택	실패·복구
Job 선언	@CpfBatchJob	value/restartable/maxConcurrentExecutions	Batch Job entry	동시 실행 제한/재시작 가능성 명시
Step 선언	@CpfBatchStep	value/order/idempotent	Job 내부 단계	재실행 시 side effect 검토
실행 정책	BatchExecutionPolicy	timeout/maxRetries/skipLimit/errorThreshold/rollbackOnFailure/checkpointRequired/pauseSupported	Job Pack 정책	Spring Batch 실제 Step 정책과 일치
Topology	BatchExecutionTopology	LOCAL/PARALLEL_STEPS/LOCAL_PARTITION	로컬 병렬/Partition	Broker remote execution 은 지원 topology 가 아님

옵션 빠른 확인

표 1-2. 옵션 빠른 확인

옵션 빠른 확인의 선택 기준과 확인 항목을 빠르게 비교합니다.

API	옵션	필수/기본 값	언제 변경	주의
@CpfBatchJob	restartable	true	재시작 금지 Job 만 false	실패 복구 설계와 일치

API	옵션	필수/기본 값	언제 변경	주의
@CpfBatchJob	maxConcurrentExecutions	1	병렬 Job 허용	원장/파일/외부 side effect 충돌 검토
@CpfBatchStep	order	0	Step 순서가 필요	중복 order 금지
@CpfBatchStep	idempotent	true	반복 호출이 안전하지 않은 Step	false 면 restart/retry 조건 엄격화

Control Plane 은 실행 요청·상태·중지·재시작·Reconcile 을 관리하고, Execution Lane 은 Lease/Fencing/Checkpoint 를 지키며 실제 Step 을 수행합니다.

선택 후 첫 구현

Batch 를 선택했다면 Job/Step 계약과 실행 Owner 를 먼저 만들고 실제 Local Runtime 에서 한 번 실행합니다.

1. 업무 단위를 Job/Step 으로 분리하고 재시작 가능한 기준점을 정합니다.
2. Local Batch Runtime 에서 대표 입력을 실행해 executionId·상태·처리 건수를 확인합니다.
3. 운영 배포에서는 Scheduler/Worker/Control Plane 역할을 Local 개발 Runtime 과 구분합니다.

2 실행 · 중지 · 재시작 · 재처리

운영 요청이 들어왔을 때 start/stop/restart/rerun/reconcile 중 무엇을 호출할지 선택합니다.

실행 요청 종류

표 2-1. 실행 요청 종류

실행 요청 종류의 선택 기준과 확인 항목을 빠르게 비교합니다.

요청	의미	언제	주의
run / scheduledRun	신규 실행	수동/스케줄	동일 업무기 중복 실행 정책
retry	실패 실행 재시도	Retry-safe 실패	같은 execution semantics 확인
restart	Checkpoint 기준 재시작	재시작 가능한 Job	restartable + checkpoint
rerun	새 실행으로 다시 처리	업무상 재처리	중복 side effect 차단
stop	실행 중지 요청	운영 중지	즉시 kill 과 동일하지 않음
onDemand	운영 즉시 실행	긴급/수동 실행	Permission/Reason/Audit

Public API 빠른 선택

표 2-2. Public API 빠른 선택

Public API 빠른 선택의 선택 기준과 확인 항목을 빠르게 비교합니다.

하려는 일	Public API / Annotation	핵심 옵션	언제 선택	실패·복구
실행 시작	BatchExecutionControlPort.start	request	Job 실행 등록	중복/lease/상태 확인
중지	BatchExecutionControlPort.stop	executionId	안전 중지	worker acknowledgment/timeout
재시작	BatchExecutionControlPort.restart	executionId	Checkpoint 재시작	stale fencing 금지
포기	BatchExecutionControlPort.abandon	executionId	복구 불가 실행 종료	감사/사유
결과 미확정	BatchExecutionControlPort.reconcile	executionId	UNKNOWN/부분 실패	실제 ledger/side effect 확인

UNKNOWN Process Kill 이나 외부 Side Effect 직후 상태가 불명확하면 성공/실패를 추정하지 않고 ledger·checkpoint·외부 상태를 reconcile 합니다.

실행·중지·복구 구현 순서

운영 액션마다 새 실행인지 기존 실행 재개인지 의미를 먼저 고정합니다.

4. Start 요청에 business key 와 idempotency key 를 연결합니다.
5. Stop/Kill 상황에서 checkpoint 가 남는지 확인합니다.
6. Restart/Rerun/Reprocess/Reconcile 을 같은 오류에 혼용하지 않고 각 선택 조건을 Test 합니다.

3 Lease · Fencing · 다중 인스턴스

동일 Job 이 여러 Worker 에서 중복 수행되지 않도록 Lease 와 fencing token 을 실행 경계마다 확인합니다.

동시성 보호

표 3-1. 동시성 보호

동시성 보호의 선택 기준과 확인 항목을 빠르게 비교합니다.

개념	개발/Runtime 계약	실패 시
Lease	실행 소유권을 TTL 기반으로 유지	갱신 실패 시 신규 side effect 중단
Fencing	현재 token 인지 side effect 전에 assert	stale worker write 차단
Checkpoint	restart 기준 상태를 durable 기록	중간 성공/실패를 추정하지 않음
Reservation/Ledger	업무키/execution 상태 예약·전이	duplicate/UNKNOWN/reconcile 근거

Public API 빠른 선택

표 3-2. Public API 빠른 선택

Public API 빠른 선택의 선택 기준과 확인 항목을 빠르게 비교합니다.

하려는 일	Public API / Annotation	핵심 옵션	언제 선택	실패·복구
현재 실행권 검증	BatchFencingPort.assertCurrent	execution/token	DB/외부 쓰기 직전	stale token 이면 실행 차단
실행 예약	BatchExecutionLedgerPort.reserve	business/execution key	중복 Job 방지	충돌이면 기존 상태 확인
상태 전이	BatchExecutionLedgerPort.transition	from/to	Lifecycle 전이	illegal transition 차단
UNKNOWN 기록	BatchExecutionLedgerPort.recordUnknown	reason	결과 미확정	reconcile queue/운영 추적

다중 Worker 검증 순서

Lease/Fencing 은 API 이름이 아니라 stale Worker 가 실제 쓰기를 못 하는지로 검증합니다.

7. 두 Worker 가 같은 Job 을 경쟁하도록 기동해 Lease Owner 가 하나인지 확인합니다.
8. Owner Worker 를 종료하고 새 Worker 가 fence token 을 갱신해 이어받는지 확인합니다.
9. 이전 Worker 가 복귀해도 stale fence token 으로 DB/Side Effect 를 갱신하지 못해야 합니다.

4 Retry · Skip · Rollback 정책

Retry/Skip 을 많이 허용하는 것이 안정성이 아니라 데이터 일관성을 유지하는 범위에서만 허용합니다.

옵션 빠른 확인

표 4-1. 옵션 빠른 확인

옵션 빠른 확인의 선택 기준과 확인 항목을 빠르게 비교합니다.

API	옵션	필수/기본 값	언제 변경	주의
BatchExecutionPolicy	timeout	기본 1 시간	SLA/업무량에 맞춤	무제한 실행 금지
BatchExecutionPolicy	maxRetries	0	Retry-safe 기술 실패	side effect 중복 위험
BatchExecutionPolicy	skipLimit	0	개별 불량 레코드 Skip 허용	무제한 Skip 금지
BatchExecutionPolicy	errorThreshold	1	오류 임계치 정책	운영 Alert 와 일치
BatchExecutionPolicy	rollbackOnFailure	true	부분 commit 이 명시 요구될 때만	원장 정합성 우선
BatchExecutionPolicy	checkpointRequired	true	stateless Job 예외	restart 필요 Job 은 true
BatchExecutionPolicy	pauseSupported	true	Pause 불필요 Runtime 예외	운영 제어 계약 확인

실패 종류별 처리

표 4-2. 실패 종류별 처리

실패 종류별 처리의 선택 기준과 확인 항목을 빠르게 비교합니다.

실패	기본 판단	Retry/Skip	복구
입력 데이터 오류	Business/data failure	Skip 정책이 있을 때만	오류 원장/재처리
DB deadlock/일시 장애	Technical	제한 Retry 가능	Tx rollback 후 재시도
외부기관 timeout after write	UNKNOWN 가능	blind retry 금지	idempotency/probe/reconcile
Worker kill	상태 미확정	새 worker 가 lease/fence 후 판단	checkpoint/ledger
Stale worker	Fencing failure	재시도보다 실행권 갱신	현재 owner 확인

정책 적용 순서

Retry·Skip·Rollback 은 오류 종류와 Side Effect 특성에 따라 Step 에 명시하고 실패 시 결과를 검증합니다.

10. Retry-safe 기술 오류만 bounded retry 대상으로 둡니다.
11. 업무 데이터 오류의 Skip 기준과 Skip limit 를 정하고 누락 건을 Evidence 로 남깁니다.
12. 외부 Side Effect 가 UNKNOWN 이면 blind retry 하지 않고 Reconcile 경로로 전환합니다.

5 Reader · Writer · Processor 와 DB

대량 I/O 를 Chunk/Partition 에 맞추고 DB3 의 Lock/Index/Transaction 차이를 함께 검토합니다.

설계 체크

표 5-1. 설계 체크

설계 체크의 선택 기준과 확인 항목을 빠르게 비교합니다.

영역	확인할 것	피해야 할 것
Reader	paging/cursor/정렬키/재시작 위치	offset-only 대량 scan, 불안정 sort
Processor	순수 변환/업무 검증/외부 side effect 분리	중복 side effect
Writer	batch size/Tx/unique key/idempotency	한 Tx 에 과도한 메모리
Partition	partition key/범위/중복 없음	경계 overlap/gap
DB3	Oracle/PostgreSQL/MariaDB datatype/index/lock	Vendor 한 곳만 검증

DB Owner Batch 도 Owner Domain DB 계약을 따릅니다. 다른 Domain 원장을 직접 읽거나 수정하지 않습니다.

6 Scheduler · Center-Cut · 운영 제어

스케줄과 긴급 Center-Cut 을 실행 API 가 아니라 권한·승인·감사 포함 운영 흐름으로 설계합니다.

운영 액션

표 6-1. 운영 액션

운영 액션의 선택 기준과 확인 항목을 빠르게 비교합니다.

액션	필수 보호	확인 Evidence
수동 실행	Permission + Reason + Audit	operator, reason, executionId
중지/재시작	Permission + 현재 상태 + Audit	before/after state
Abandon	강한 권한 + Reason + Approval	승인/사유/결과
Reconcile	상태 조회 + 안전 probe	UNKNOWN→final transition
Center-Cut	실행 범위/승인/추적	대상/건수/결과/실패

Center-Cut 실행 전 확인

표 6-2. Center-Cut 실행 전 확인

Center-Cut 실행 전 확인의 선택 기준과 확인 항목을 빠르게 비교합니다.

확인	왜 필요한가	Evidence
대상 범위	segment/job parameter 오지정 방지	target/parameter
권한·사유·승인	위험 실행의 실행 근거 확보	operator/reason/approval
Lease · Fence	stale worker 중복 실행 차단	lease/fence owner
Checkpoint · 복구 계획	중지 후 재개 지점과 rollback 판단	checkpoint/recovery plan

운영 제어 연결 순서

Scheduler·Center-Cut·Control Plane 은 실행 역할과 승인 경계를 분리한 뒤 운영자가 실제 조작할 경로를 확인합니다.

- Schedule 등록/변경은 권한·사유·승인·Audit 를 연결합니다.
- Center-Cut 은 실행 대상/Host/명령 유형을 제한하고 실행 전 dry-run 또는 검증 절차를 둡니다.
- Control Plane 장애 시 Worker 실행과 복구 상태가 어떻게 보존되는지 확인합니다.

7 Batch 검증

정상 완료뿐 아니라 Kill·중복 실행·Lease 만료·DB 장애·외부 UNKNOWN 을 포함해 재실행 결과까지 검증합니다.

필수 Runtime 시나리오

표 7-1. 필수 Runtime 시나리오

필수 Runtime 시나리오의 선택 기준과 확인 항목을 빠르게 비교합니다.

시나리오	기대 결과	Evidence
정상 run	COMPLETED + expected records	execution/step/DB
동일 키 동시 run	하나만 실행권 획득	reservation/lease
Worker kill	stale worker write 0	fence/checkpoint
DB transient failure	정책 범위 retry/rollback	attempt/tx
외부 UNKNOWN	결과 추정 없이 reconcile	recoveryId/ledger
restart	checkpoint 이후 정확히 재개	before/after counts
rerun	중복 side effect 없음	idempotency key

Batch 변경 후 검증

```
./gradlew cpfVerifyTargeted -PcpfTargetCapabilities=batch
./gradlew cpfVerifyFullLocal
```

8 Control Plane 과 독립 Runtime

개발 편의용 Local Runtime 과 운영 배포 Process 를 구분하고 각 실행 역할의 Owner 와 상태 계약을 맞춥니다.

Control Plane·Scheduler·Worker

Control Plane 은 실행 요청/상태/복구를 관리하고 Scheduler 는 실행 시점을 결정하며 Worker 는 실제 Job/Step 을 수행합니다. 운영에서는 각 역할을 독립 Process 로 배치할 수 있습니다.

Center-Cut-Agent·Job Pack

Center-Cut 은 대량 대상을 분할·통제하는 실행 경계이고 Agent 는 실행 노드와 Control Plane 을 연결합니다. 배포된 Job Pack 은 실행 코드와 정책/버전 식별자를 함께 추적해야 합니다.

개발용 Local Runtime 과 운영 Process

cpf-local-batch-runtime 은 개발 시 여러 역할을 독립 ApplicationContext 로 띄우는 도구입니다. 운영 Topology 를 한 JVM 으로 축약했다는 근거로 사용하지 않습니다.

9 대량 처리·운영 인계·Source

Chunk/Partition 크기는 처리량뿐 아니라 Lock, 재시작 지점, 메모리, 외부 Side Effect 를 함께 고려합니다.

Partition/Chunk 용량 결정

한 Chunk 가 너무 크면 Rollback 범위와 Lock 시간이 커지고, 너무 작으면 commit/네트워크 오버헤드가 커집니다. 실제 데이터 분포와 DB3 Query Plan 을 기준으로 결정합니다.

운영 인계 항목

jobName, executionId, business key, checkpoint, lease/fencing 정책, restart/rerun/reconcile 선택 조건, 외부 Side Effect 식별자, Runtime 검증 명령과 Evidence 위치를 인계합니다.

대량 처리 설계 판단

설계 항목	확인 기준	운영 Evidence	피해야 함
Chunk 크기	Rollback 범위·commit 비용·메모리	처리량/commit 시간/rollback 시간	건수만 보고 고정
Partition 수	DB Lock·Connection·외부기관 허용량	동시 실행 수/DB wait/기관 limit	CPU 수만큼 무조건 확대
Checkpoint	재시작 비용·Side Effect 경계	checkpoint 위치/재개 건수	마지막 성공 위치 불명
Lease/Fencing	stale worker 쓰기 차단	owner/fence token/expiry	heartbeat 만 믿고 중복 허용
운영 인계	restart/rerun/reprocess/reconcile 조건	executionId/recoveryId/판단 근거	“실패 시 재실행”만 전달

Source: [cpf-batch/api](#) · [cpf-batch/runtime](#) · [cpf-batch/control-plane](#) · [cpf-batch/scheduler](#) · [cpf-batch/worker](#) · [cpf-batch/center-cut-runtime](#)

대량 처리 검증 순서

Chunk·Partition 값을 정한 뒤 처리량만 보지 않고 재시작 가능성과 중복 방지까지 같은 실행에서 확인합니다.

- 대표 데이터로 정상 처리량·commit 주기·DB wait 를 먼저 측정하고 checkpoint 위치를 기록합니다.
- Worker 또는 외부 연계를 중간에 실패시켜 Lease/Fencing 이 stale 실행의 쓰기를 차단하는지 확인합니다.
- Restart/Rerun/Reprocess/Reconcile 중 선택한 복구 경로로 재개하고 처리 건수와 Side Effect 중복이 없는지 비교합니다.
- executionId·recoveryId·checkpoint-owner/fence token 과 최종 결과를 운영 Evidence 로 남깁니다.

Source: cpf-batch/api · cpf-batch/runtime · cpf-batch/control-plane · cpf-batch/scheduler · cpf-batch/worker · cpf-batch/center-cut

운영 인계 완료 확인

배치 개발 완료는 처리량 숫자만이 아니라 재시작·중복방지·운영자가 판단할 Evidence 까지 전달됐을 때 달성됩니다.

- 정상 실행 1 회와 중간 실패 1 회를 같은 Job/Business Key 기준으로 비교해 checkpoint 와 executionId 변화가 의도한 정책과 일치하는지 확인합니다.
- Lease/Fencing/Heartbeat 상태를 Worker 교체·Process 종료 상황에서 확인하고 stale Worker 가 쓰기를 이어가지 못하는지 검증합니다.
- Restart/Rerun/Reprocess/Reconcile 중 허용 경로와 금지 경로를 Runbook 에 구분하고 외부 Side Effect 의 idempotency key 또는 recoveryId 를 함께 인계합니다.
- Control Plane·Scheduler·Worker·Center-Cut/Agent 의 실제 배포 역할과 Local Batch Runtime 개발 도구의 역할을 혼동하지 않도록 실행 위치와 로그·Evidence 경로를 남깁니다.

10 Runtime 역할과 실패를 함께 설계하기

Batch Job/Step 구현은 정상 실행 코드만으로 끝나지 않습니다. Control Plane·Scheduler·Worker·Center-Cut·Agent 가 어떤 상태를 소유하고 실패 후 누가 이어받는지까지 개발 단계에서 달성됩니다.

역할별 Owner

- Control Plane — Policy·Schedule·Deployment·Recovery 와 운영 제어를 소유합니다.
- Scheduler — Trigger 와 실행 시점을 결정하고 같은 실행을 중복 발행하지 않도록 상태를 관리합니다.
- Worker — 실제 Step/Chunk/Partition 을 실행하고 Lease·Heartbeat·Checkpoint 를 갱신합니다.
- Center-Cut/Agent — Segment·Job Pack·원격 실행을 연결하되 Business Domain 의 업무 Master 를 소유하지 않습니다.

Process Kill 과 Takeover

Worker 가 응답하지 않는다고 즉시 새 Worker 가 같은 데이터를 쓰게 하지 않습니다. Lease 만료, Heartbeat, Fencing Token, Checkpoint 를 함께 보고 이전 Worker 가 더 이상 유효한 Write 를 할 수 없는지 확인한 뒤 Takeover 합니다.

- Kill 직전/직후 Checkpoint 와 DB 반영 경계를 확인합니다.
- 오래된 Fencing Token 의 Write 가 실제로 거부되는지 검증합니다.
- Restart/Rerun/Reprocess/Reconcile 을 같은 “재실행”으로 취급하지 않습니다.

개발 완료 검증

최소 Runtime 검증 Fresh 상태 → 정상 실행 → 중간 실패 → Process Kill → Lease/Fencing → Takeover → Checkpoint 재개 → 중복 실행 방지 → 외부 Side Effect UNKNOWN/Reconcile → DB/Trace/Audit 확인 → 재실행

Source: cpf-batch/control-plane · cpf-batch/scheduler · cpf-batch/worker · cpf-batch/center-cut · cpf-batch/agent